

WEEK-11

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-21

Task1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
saivihamsh@Varshini-PC ~/OSSP/skill/week10/task2$ cd ~/OSSP/skill
saivihamsh@Varshini-PC ~/OSSP/skill$ mkdir -p week11/task1
saivihamsh@Varshini-PC ~/OSSP/skill$ cd week11/task1
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ nano calculator.h
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ nano calculator.c
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ nano interface.h
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ nano interface.c
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ nano main.c
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ gcc_main.c calculator.c
interface.c -o T1
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ ls -l
total 44
-rwxr-xr-x 1 madhulika madhulika 16416 Sep 25 08:46 T1
-rw-r--r-- 1 madhulika madhulika 290 Sep 25 08:30 calculator.c
-rw-r--r-- 1 madhulika madhulika 177 Sep 25 08:28 calculator.h
-rw-r--r-- 1 madhulika madhulika 673 Sep 25 08:31 interface.c
-rw-r--r-- 1 madhulika madhulika 113 Sep 25 08:31 interface.h
-rw-r--r-- 1 madhulika madhulika 2116 Sep 25 08:32 main.c
-rw-r--r-- 1 madhulika madhulika 2018 Sep 19 03:07 race_counter.c
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ ./T1
=====
          SKILL WEEK 11 - TASK 1
          MODULE INTEGRATION
=====

=====
          MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 1
Enter first number: 10
Enter second number: 20

Result: 10 + 20 = 30
```

```
=====
          MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 2
Enter first number: 50
Enter second number: 30

Result: 50 - 30 = 20

=====
          MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 3
Enter first number: 5
Enter second number: 6

Result: 5 * 6 = 30
```

```
=====
MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 3
Enter first number: 5
Enter second number: 6

Result: 5 * 6 = 30
```

```
=====
MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 4
Enter first number: 20
Enter second number: 5

Result: 20 / 5 = 4
```

```
=====
MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 4
Enter first number: 20
Enter second number: 0

Error: Division by zero is not allowed.
```

```
=====
MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 9

Invalid choice. Please select 1 to 5.
```

```
=====
MODULAR CALCULATOR
=====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
=====
Enter your choice: 5

Exiting program successfully.
```

```
GNU nano 8.7.1 calculator.h *
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
int divide_numbers(int a, int b, int *result);

#endif
```

```
GNU nano 8.7.1 calculator.c *
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

int divide_numbers(int a, int b, int *result)
{
    if (b == 0)
        return 0;

    *result = a / b;
    return 1;
}
```

```
GNU nano 8.7.1 interface.h *
#ifndef INTERFACE_H
#define INTERFACE_H

void display_menu(void);
int read_integer(const char *message);

#endif
```

```
GNU nano 8.7.1 interface.c *
#include <stdio.h>
#include "interface.h"

void display_menu(void)
{
    printf("\n=====\n");
    printf("          MODULAR CALCULATOR\n");
    printf("=====\n");
    printf("1. Addition\n");
    printf("2. Subtraction\n");
    printf("3. Multiplication\n");
    printf("4. Division\n");
    printf("5. Exit\n");
    printf("=====\n");
}

int read_integer(const char *message)
{
    int value;

    printf("%s", message);

    if (scanf("%d", &value) != 1) {
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF)
            ;
        return -1;
    }

    return value;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
GNU nano 8.7.1 main.c *
#include <stdio.h>
#include "calculator.h"
#include "interface.h"

int main()
{
    int choice;
    int a, b;
    int result;

    printf("=====\n");
    printf("          SKILL WEEK 11 - TASK 1\n");
    printf("          MODULE INTEGRATION\n");
    printf("=====\n");

    while (1)
    {
        display_menu();

        choice = read_integer("Enter your choice: ");

        if (choice == -1)
        {
            printf("\nInvalid input. Please enter a number.\n");
            continue;
        }

        if (choice == 5)
        {
            printf("\nExiting program successfully.\n");
            break;
        }

        if (choice < 1 || choice > 5)
        {
            printf("\nInvalid choice. Please select 1 to 5.\n");
            continue;
        }
    }
}
```

```
GNU nano 8.7.1 main.c *
}

switch (choice)
{
    case 1:
        result = add(a, b);
        printf("\nResult: %d + %d = %d\n",
            a, b, result);
        break;

    case 2:
        result = subtract(a, b);
        printf("\nResult: %d - %d = %d\n",
            a, b, result);
        break;

    case 3:
        result = multiply(a, b);
        printf("\nResult: %d * %d = %d\n",
            a, b, result);
        break;

    case 4:
        if (divide_numbers(a, b, &result))
        {
            printf("\nResult: %d / %d = %d\n",
                a, b, result);
        }
        else
        {
            printf("\nError: Division by zero is not allowed.\n");
        }
        break;
}

return 0;
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

Observation: The modular program was successfully implemented by separating the calculator operations, user interface, and main control logic into independent modules. The modules were connected through header files and their interfaces were validated during compilation and execution. End-to-end testing was performed for addition, subtraction, multiplication, and division. Error handling was also verified by testing division by zero and invalid menu choices. The modular structure improved reliability and made the program easier to understand, maintain, and modify.

Task2: To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```

saivihamsh@Varshini-PC ~/OSSP/skill/week11/task1$ cd ~/OSSP/skill/week11
saivihamsh@Varshini-PC ~/OSSP/skill/week11$ mkdir -p task2
saivihamsh@Varshini-PC ~/OSSP/skill/week11$ cd task2
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task2$ nano error_handler.c
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task2$ gcc error_handler.c -o
T2
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task2$ ./T2
=====
                SKILL WEEK 11 - TASK 2
                ERROR HANDLING SYSTEM
=====

Type 'help' to display available commands.

command> help

Available commands:
add      - Add two numbers
divide   - Divide two numbers
help     - Display available commands
exit     - Exit the program

command> divide
Enter two numbers: 20
0
ERROR: Runtime error: Division by zero.

command> ERROR: Empty command entered.

command> multiply
ERROR: Invalid syntax or unknown command.

command> add
Enter two numbers: abc 10
ERROR: Invalid numeric input.

command> exit
Program terminated gracefully.
saivihamsh@Varshini-PC ~/OSSP/skill/week11/task2$

```

```

GNU nano 8.7.1 error_handler.c *
printf("ERROR: %s\n", message);
log_error(message);
}

int validate_command(char *command)
{
    if (strcmp(command, "add") == 0 ||
        strcmp(command, "divide") == 0 ||
        strcmp(command, "help") == 0 ||
        strcmp(command, "exit") == 0)
    {
        return 1;
    }

    return 0;
}

void clear_input_buffer()
{
    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF)
        ;
}

void process_command(char *command)
{
    int a, b;

    if (strcmp(command, "add") == 0)
    {
        printf("Enter two numbers: ");

        if (scanf("%d %d", &a, &b) != 2)
        {
            show_error("Invalid numeric input.");
            clear_input_buffer();
        }
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

```
GNU nano 8.7.1 error_handler.c *
char command[MAX_INPUT];

printf("=====\n");
printf("      SKILL WEEK 11 - TASK 2\n");
printf("      ERROR HANDLING SYSTEM\n");
printf("=====\n");

printf("\nType 'help' to display available commands.\n");

while (1)
{
    printf("\ncommand> ");

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        show_error("Input reading failed.");
        break;
    }

    command[strcspn(command, "\n")] = '\0';

    /* Empty command */
    if (strlen(command) == 0)
    {
        show_error("Empty command entered.");
        continue;
    }

    /* Exit */
    if (strcmp(command, "exit") == 0)
    {
        printf("Program terminated gracefully.\n");
        break;
    }

    /* Invalid command / syntax */
    if (!validate_command(command))
    |
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Observation: The error handling system was successfully implemented to detect runtime errors, invalid commands, and incorrect input. Appropriate error messages were generated for different failure scenarios, and the errors were recorded in a log file with timestamps. Division by zero and invalid numeric input were handled without terminating the program, allowing execution to continue normally. Invalid syntax was also detected and reported, demonstrating graceful recovery and reliable error handling.

SESSION-22

Task1: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
saiviamsh@Varshini-PC ~/OSSP/skill/week11/task2$ cd ~/OSSP/skill
saiviamsh@Varshini-PC ~/OSSP/skill$ mkdir -p week12/task1
saiviamsh@Varshini-PC ~/OSSP/skill$ cd week12/task1
saiviamsh@Varshini-PC ~/OSSP/skill/week12/task1$ valgrind --version
valgrind-3.26.0
saiviamsh@Varshini-PC ~/OSSP/skill/week12/task1$ nano memory_test.c
saiviamsh@Varshini-PC ~/OSSP/skill/week12/task1$ gcc -g memory_test.c -o
T1
saiviamsh@Varshini-PC ~/OSSP/skill/week12/task1$ ./T1
=====
                SKILL WEEK 12 - TASK 1
                VALGRIND MEMORY TEST
=====
Name: Valgrind Memory Test
Numbers: 10 20 30 40 50

Memory test completed.
saiviamsh@Varshini-PC ~/OSSP/skill/week12/task1$ valgrind --leak-check=f
ull --track-origins=yes ./T1
==2611== Memcheck, a memory error detector
==2611== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2611== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2611== Command: ./T1
==2611==
=====
                SKILL WEEK 12 - TASK 1
                VALGRIND MEMORY TEST
=====
Name: Valgrind Memory Test
Numbers: 10 20 30 40 50

Memory test completed.
```

```
==2611==
==2611== HEAP SUMMARY:
==2611==   in use at exit: 70 bytes in 2 blocks
==2611==   total heap usage: 3 allocs, 1 frees, 1,094 bytes allocated
==2611==
==2611== 20 bytes in 1 blocks are definitely lost in loss record 1 of 2
==2611==   at 0x4850858: malloc (vg_replace_malloc.c:447)
==2611==   by 0x40011CC: create_memory (memory_test.c:11)
==2611==   by 0x400131A: main (memory_test.c:50)
==2611==
==2611== 50 bytes in 1 blocks are definitely lost in loss record 2 of 2
==2611==   at 0x4850858: malloc (vg_replace_malloc.c:447)
==2611==   by 0x40011BE: create_memory (memory_test.c:10)
==2611==   by 0x400131A: main (memory_test.c:50)
==2611==
==2611== LEAK SUMMARY:
==2611==   definitely lost: 70 bytes in 2 blocks
==2611==   indirectly lost: 0 bytes in 0 blocks
==2611==   possibly lost: 0 bytes in 0 blocks
==2611==   still reachable: 0 bytes in 0 blocks
==2611==   suppressed: 0 bytes in 0 blocks
==2611==
==2611== For lists of detected and suppressed errors, rerun with: -s
==2611== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week12/task1$ nano memory_test.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week12/task1$ gcc -g memory_test.c -o
T1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week12/task1$ ./T1
=====
                SKILL WEEK 12 - TASK 1
                VALGRIND MEMORY TEST
=====
Name: Valgrind Memory Test
Numbers: 10 20 30 40 50
Allocated memory released successfully.

Memory test completed.
```

```

GNU nano 8.7.1                                memory_test.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void create_memory()
{
    char *name;
    int *numbers;

    name = malloc(50 * sizeof(char));
    numbers = malloc(5 * sizeof(int));

    if (name == NULL || numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    strcpy(name, "Valgrind Memory Test");

    for (int i = 0; i < 5; i++)
    {
        numbers[i] = (i + 1) * 10;
    }

    printf("Name: %s\n", name);
    printf("Numbers: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    /*

```

```

    * Memory is intentionally not released here.
    * Valgrind should detect the memory leak.
    */
}

int main()
{
    printf("=====\n");
    printf("        SKILL WEEK 12 - TASK 1\n");
    printf("        VALGRIND MEMORY TEST\n");
    printf("=====\n");

    create_memory();

    printf("\nMemory test completed.\n");

    return 0;
}

```

Observation: The memory management program was successfully tested using Valgrind. Initially, Valgrind identified memory leaks of 70 bytes in two blocks because dynamically allocated memory was not released. The memory management code was then corrected by using free() to release the allocated resources. The program was tested again to verify proper memory cleanup and memory management.

Task2: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```
Memory test completed.
saivihamsh@Varshini-PC ~/OSSP/skill/week12/task1$ cd ~/OSSP/skill/week12
saivihamsh@Varshini-PC ~/OSSP/skill/week12$ mkdir -p task2
saivihamsh@Varshini-PC ~/OSSP/skill/week12$ cd task2
saivihamsh@Varshini-PC ~/OSSP/skill/week12/task2$ nano calculator.c
saivihamsh@Varshini-PC ~/OSSP/skill/week12/task2$ gcc_calculator.c -o calculator
saivihamsh@Varshini-PC ~/OSSP/skill/week12/task2$ ./calculator 10 + 20
30
saivihamsh@Varshini-PC ~/OSSP/skill/week12/task2$ nano test_script.sh
saivihamsh@Varshini-PC ~/OSSP/skill/week12/task2$ chmod +x test_script.sh
saivihamsh@Varshini-PC ~/OSSP/skill/week12/task2$ ./test_script.sh
```

```
=====
SKILL WEEK 12 - TASK 2
AUTOMATED TEST REPORT
=====

Test 1: Addition
Command : ./calculator 10 + 20
Expected: 30
Actual : 30
Result : PASS
-----

Test 2: Subtraction
Command : ./calculator 50 - 20
Expected: 30
Actual : 30
Result : PASS
-----

Test 3: Multiplication
Command : ./calculator 5 '*' 6
Expected: 30
Actual : 30
Result : PASS
-----

Test 4: Division
Command : ./calculator 20 / 5
Expected: 4
Actual : 4
Result : PASS
```

```
Result : PASS
-----

Test 5: Division by Zero
Command : ./calculator 20 / 0
Expected: ERROR: Division by zero
Actual : ERROR: Division by zero
Result : PASS
-----

Test 6: Invalid Operator
Command : ./calculator 10 '%' 2
Expected: ERROR: Invalid operator
Actual : ERROR: Invalid operator
Result : PASS
-----

=====
TOTAL TESTS : 6
PASSED : 6
FAILED : 0
=====
```

```
GNU nano 8.7.1 calculator.c *
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int a, b;
    char operation;

    if (argc != 4)
    {
        printf("Usage: %s <number1> <operator> <number2>\n", argv[0]);
        return 1;
    }

    a = atoi(argv[1]);
    operation = argv[2][0];
    b = atoi(argv[3]);

    switch (operation)
    {
        case '+':
            printf("%d\n", a + b);
            break;

        case '-':
            printf("%d\n", a - b);
            break;

        case '*':
            printf("%d\n", a * b);
            break;

        case '/':
            if (b == 0)
            {
                printf("ERROR: Division by zero\n");
                return 2;
            }
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
GNU nano 8.7.1 test_script.sh *
#!/bin/bash

PROGRAM="./calculator"
REPORT="test_report.txt"

total=0
passed=0
failed=0

echo "===== " > "$REPORT"
echo "      SKILL WEEK 12 - TASK 2" >> "$REPORT"
echo "      AUTOMATED TEST REPORT" >> "$REPORT"
echo "===== " >> "$REPORT"
echo "" >> "$REPORT"

run_test()
{
    test_name="$1"
    command="$2"
    expected="$3"

    total=$((total + 1))

    actual=$(eval "$command" 2>&1)

    echo "Test $total: $test_name" >> "$REPORT"
    echo "Command : $command" >> "$REPORT"
    echo "Expected: $expected" >> "$REPORT"
    echo "Actual   : $actual" >> "$REPORT"

    if [ "$actual" = "$expected" ]
    then
        echo "Result   : PASS" >> "$REPORT"
        passed=$((passed + 1))
    else
        echo "Result   : FAIL" >> "$REPORT"
        failed=$((failed + 1))
    fi
}
```

```

    echo "Result : FAIL" >> "$REPORT"
    failed=$((failed + 1))
fi

echo "-----" >> "$REPORT"
}

run_test "Addition" "./calculator 10 + 20" "30"
run_test "Subtraction" "./calculator 50 - 20" "30"
run_test "Multiplication" "./calculator 5 '*' 6" "30"
run_test "Division" "./calculator 20 / 5" "4"
run_test "Division by Zero" "./calculator 20 / 0" "ERROR: Division by zero"
run_test "Invalid Operator" "./calculator 10 '%' 2" "ERROR: Invalid operato>

echo "" >> "$REPORT"
echo "===== " >> "$REPORT"
echo "TOTAL TESTS : $total" >> "$REPORT"
echo "PASSED      : $passed" >> "$REPORT"
echo "FAILED       : $failed" >> "$REPORT"
echo "===== " >> "$REPORT"

cat "$REPORT"

```

Observation: The automated testing system was successfully implemented using a C program and a shell test script. Multiple test cases were designed for addition, subtraction, multiplication, division, division by zero, and invalid operators. The test script automatically executed each case, compared the actual output with the expected output, and recorded the result as PASS or FAIL. A test report was generated containing the test cases, expected outputs, actual outputs, and overall test statistics.